

Immutable Classes

Rules for Immutable classes:

- Mark my class as final.
- Mark my instance variable as final.
- Mark my instance variable as private.
- No setters.
- Defensive copy in getters/constructors→non-primitive ko alag se create kro or uska refrence de do.
- Immutable objects are important in threads.

```
public class ImmutableClasses2 {
    public static void main(String[] args) {
        College college = new College("GDC Solan", "Himachal Pradesh");
        Student s1 = new Student(28, "Priyansh Sharma", college);
        System.out.println(s1.getCollege().name); // GDC Solan
        s1.getCollege().name = "IIT Solan"; // if we change the name of college.
        System.out.println(s1.getCollege().name); // IIT Solan.
    }
}

// purely Immutable class.
// defensive copy of non-primitive (college).
final class Student{
    private final int age;
    private final String name;
    private final College college;

    Student(int age, String name, College college){
        this.age = age;
        this.name = name;
        this.college = new College(college.name, college.address); // defensive copy.
    }
    // getters
    public int getAge(){
        return this.age;
    }
    public String getName(){
        return this.name;
    }
    public College getCollege(){
        return new College(this.college.name, this.college.address); //defensive copy/deep copy.
    }
}

class College{ // normal/mutable class.
    String name;
    String address;

    College(String name, String address){
        this.name = name;
        this.address = address;
    }
}
```